SLE-3: ARCHITECTURAL DESIGN USING FULL C4 MODEL

Course: 02AML204 – Introduction to Artificial Intelligence

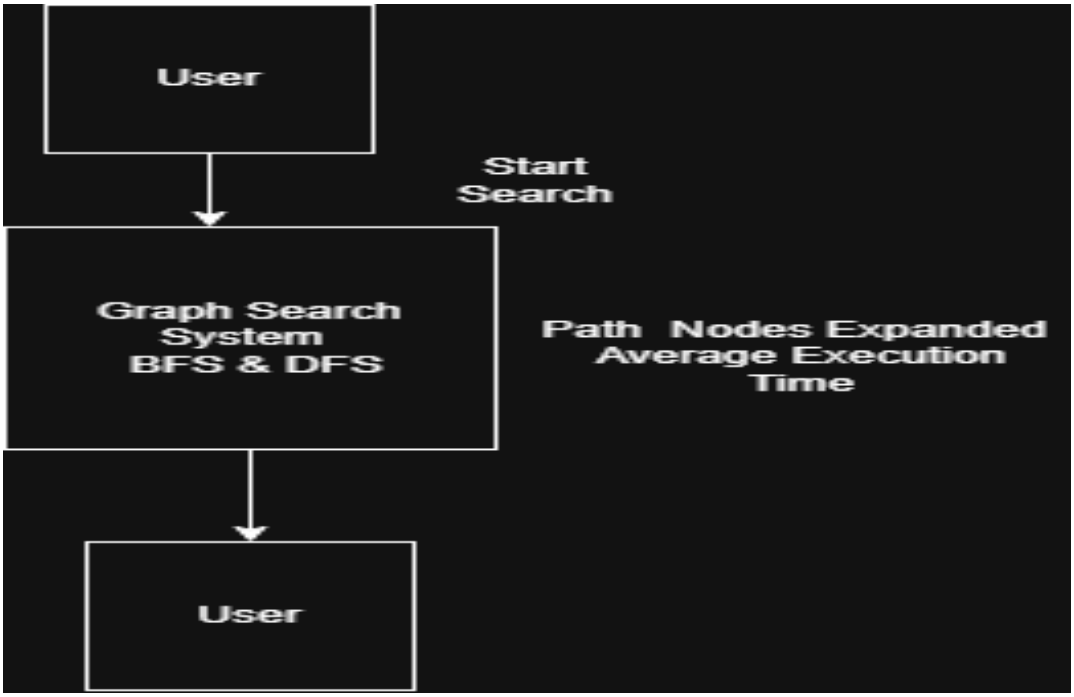
PRN	26UAM301
Name	Sneha Rahul Patil
Division	A
System	Graph Search System using BFS and DFS
GitHub Link	https://github.com/snehapatil2811/IAI-SLE3-26UAM301.git

1. System Title & Short Description

System Title: Graph Search System using BFS and DFS

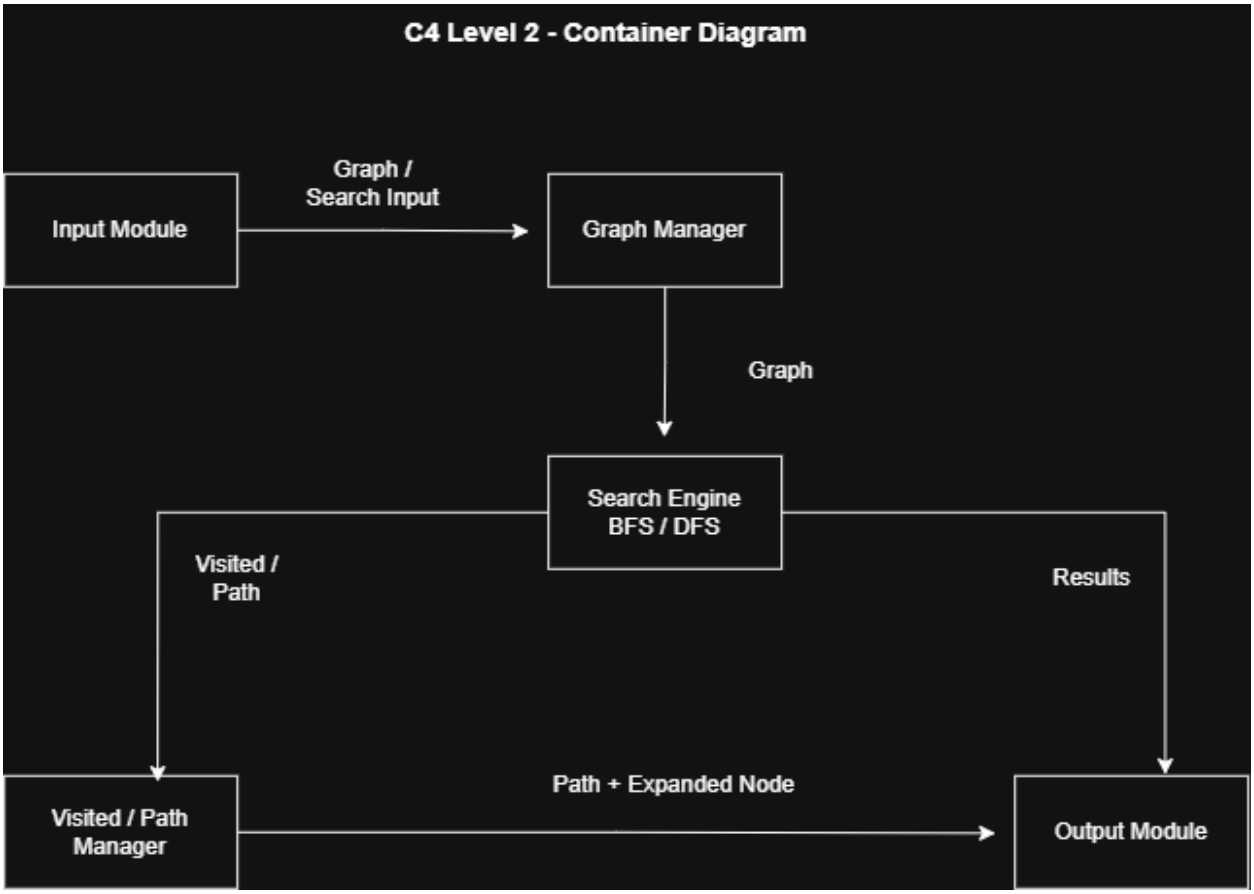
I developed a graph search system that implements two search algorithms: Breadth-First Search (BFS) and Depth-First Search (DFS). The system uses a predefined graph with a starting node A and a goal node BC. It finds a path from the start node to the goal node and counts the number of expanded nodes. I also used Python's timeit module to measure the average execution time of BFS and DFS over 10,000 runs. The system is designed to compare the performance of both search algorithms on the same graph.

2. Context Diagram (Level 1)



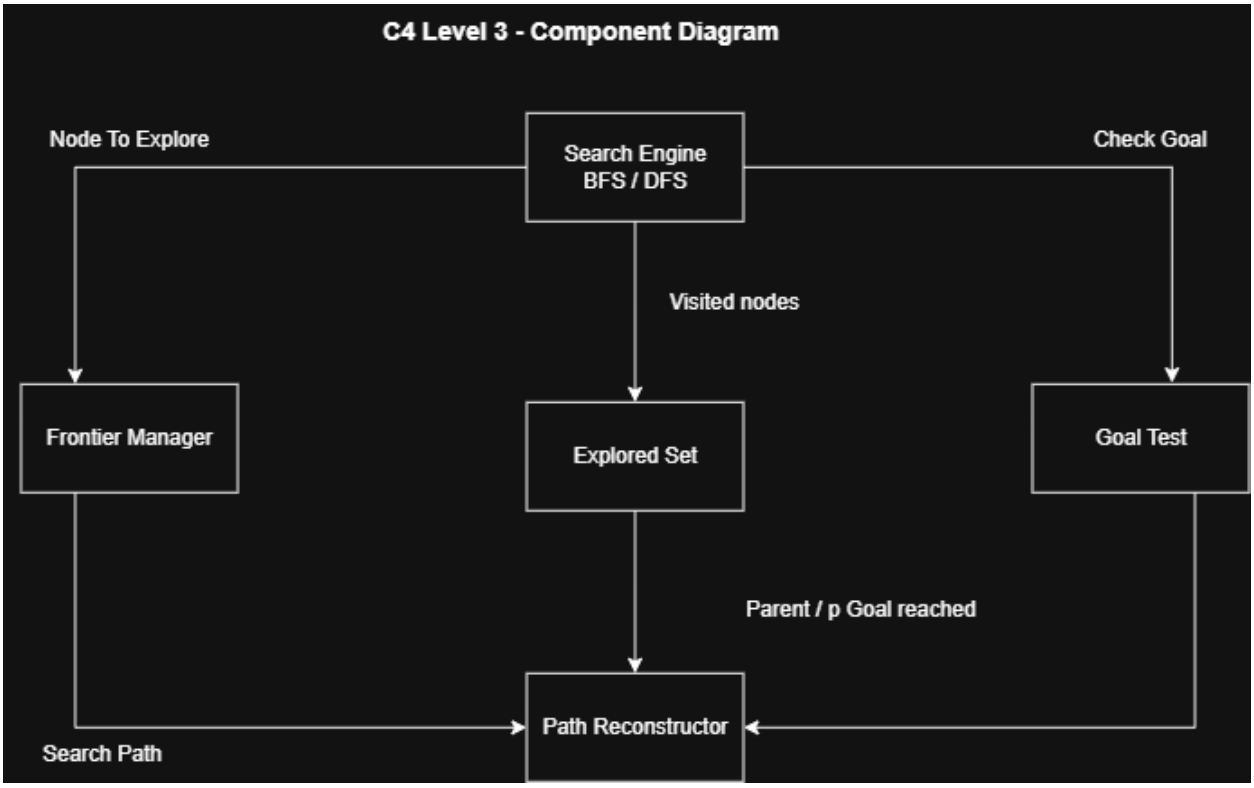
I use the Graph Search System to perform BFS and DFS on a predefined graph. I provide the search operation through the system, which uses the starting node A and goal node BC. The system performs the selected search algorithm and returns the path, number of expanded nodes, and average execution time as the output.

3. Container Diagram (Level 2)



The Container Diagram divides my system into five main parts. The Input Module handles the search request and graph information. The Graph Manager represents the predefined graph structure used by both algorithms. The Search Engine runs BFS or DFS. The Visited/Path Manager represents the tracking of visited nodes and path information. The Output Module presents the final path, expanded-node count, and execution-time result.

4. Component Diagram (Level 3)



I selected the Search Engine as the main container for the component-level view. The Frontier Manager handles the nodes waiting to be explored using the queue or stack behavior of BFS and DFS. The Explored Set tracks visited nodes. The Goal Test checks whether the current node is the required goal node BC. The Path Reconstructor represents the path information maintained while searching.

5. Code Level Overview (Level 4)

GRAPH : Stores the predefined graph and its node connections.

bfs() : Performs Breadth-First Search using a queue and visited set.

dfs() : Performs Depth-First Search using a stack and visited set.

run_algorithm() : Selects BFS or DFS based on the algorithm name.

average_time() : Uses timeit to calculate average execution time.

main program : Runs both algorithms and displays path, expanded nodes, and timing.

The code-level overview contains only the main data structure and functions, rather than the complete source code.

6. Design Decisions

I separated the architecture into input, graph, search, path/visited tracking, and output responsibilities so that each part has a clear purpose. I kept BFS and DFS inside the Search Engine because they perform the main search operation. I use visited-node tracking to avoid unnecessary repeated exploration, and I use the same graph for both algorithms to make the performance comparison consistent.

7. AI Contribution Note

AI tools used: ChatGPT.

What AI helped with: I used AI for guidance in understanding the C4 Model, organizing the architecture, preparing explanations, and formatting the report.

What I did myself: I developed and tested my BFS and DFS Python program, used the actual project structure for the architecture, checked the program results, and prepared my final project based on my implementation.

8. Conclusion

I learned how to represent the same software system at different architectural levels using the C4 Model. The Context level shows the system and its user, while the Container level shows the main building blocks. The Component level explains the internal parts of the Search Engine, and the Code level connects the architecture to my actual Python functions. This helped me understand how my BFS and DFS program can be represented as a complete software architecture.